Persisting Conversations with a SQLite Checkpointer

Replacing the in-memory checkpointer with a SQLite-backed one so conversations survive app restarts and page reloads — and wiring the Streamlit frontend to load past threads on startup.

WHAT THIS CHAPTER COVERS

- Why InMemorySaver loses all history on restart
- The three LangGraph checkpointer types
- Installing langgraph-checkpoint-sqlite
- Creating a SQLite connection with check_same_thread=False
- Wiring SqliteSaver into graph.compile()
- How one invoke() creates three checkpoints
- Inspecting chatbot.db with a SQLite viewer
- retrieve_all_threads() and frontend thread-list bootstrapping

Persisting Conversations with a SQLite Checkpointer

RECAP — CHAPTER 13

Chapter 13 added a *multi-thread "New Chat" / resume UI* in Streamlit, letting a user switch between separate conversation threads and reload any thread's message history mid-session. That version, however, still used **InMemorySaver** under the hood — so every thread lived only in RAM. This chapter replaces that in-memory checkpointer with a persistent, disk-backed one so conversations are never lost.

1. The Problem: No Permanent Storage

Definition

Up to Chapter 13, the chatbot's short-term memory was implemented with **InMemorySaver**, LangGraph's RAM-based checkpointer. Every message exchanged with the user — across every thread — was held only in the running process's memory.

Problem It Solves (What This Chapter Fixes)

Because InMemorySaver stores everything in RAM, closing the application or simply reloading the page wipes out *all* conversation history for *every* thread. There is no way to come back three or four days later and resume a conversation from where it was left — the moment the process ends, the memory is empty again.

IMPORTANT

InMemorySaver is not "broken" — it is *working exactly as designed*. RAM-based storage is appropriate for short-lived sessions and quick testing, but it was never meant to survive a process restart. The fix is not to patch InMemorySaver, but to swap in a checkpointer backed by durable storage.

The solution: replace `InMemorySaver` with a *database-backed checkpointer*. Every message the user sends and every reply the AI generates gets written to an on-disk database. When the app restarts, the checkpointer reads directly from that database, and the conversation resumes exactly where it left off.

↻ Quick Revision — The Problem

- InMemorySaver stores all thread state in RAM only.
- Closing the app or reloading the page clears all conversation history.
- There is no permanent storage with InMemorySaver — this is the core limitation being solved in this chapter.
- The fix: swap the checkpointer for one backed by a database file on disk.

2. LangGraph's Three Checkpointer Types

Definition

LangGraph's documentation defines three built-in checkpointer backends, each suited to a different stage of an application's lifecycle.

Figure 14.1 — The three LangGraph checkpointer backends and when each is appropriate.

Checkpointer	Storage	Typical Use Case
InMemorySaver	RAM	Quick testing; not persistent across restarts
SqliteSaver	Local SQLite file	Prototyping / learning — small, file-based, easy to inspect
PostgresSaver	PostgreSQL database	Production-grade applications

This chapter uses **SqliteSaver** because the project is still in a learning phase: SQLite requires no separate server, stores everything in a single `.db` file, and is easy to inspect with a lightweight viewer — while still proving the persistence pattern that production code (with PostgresSaver) would follow.

↻ Quick Revision — Checkpointer Types

- InMemorySaver: RAM-based, resets on every restart.
- SqliteSaver: file-based SQLite database, ideal for prototyping and learning.
- PostgresSaver: full production-grade database checkpointer.
- SqliteSaver is not built into core LangGraph — it ships in a separate community package, langgraph-checkpoint-sqlite.

3. Setting Up the SQLite Checkpointer (Backend)

How It Works

Wiring a SQLite-backed checkpointer into an existing LangGraph app takes five concrete steps, shown below in the order they are applied to the backend file.

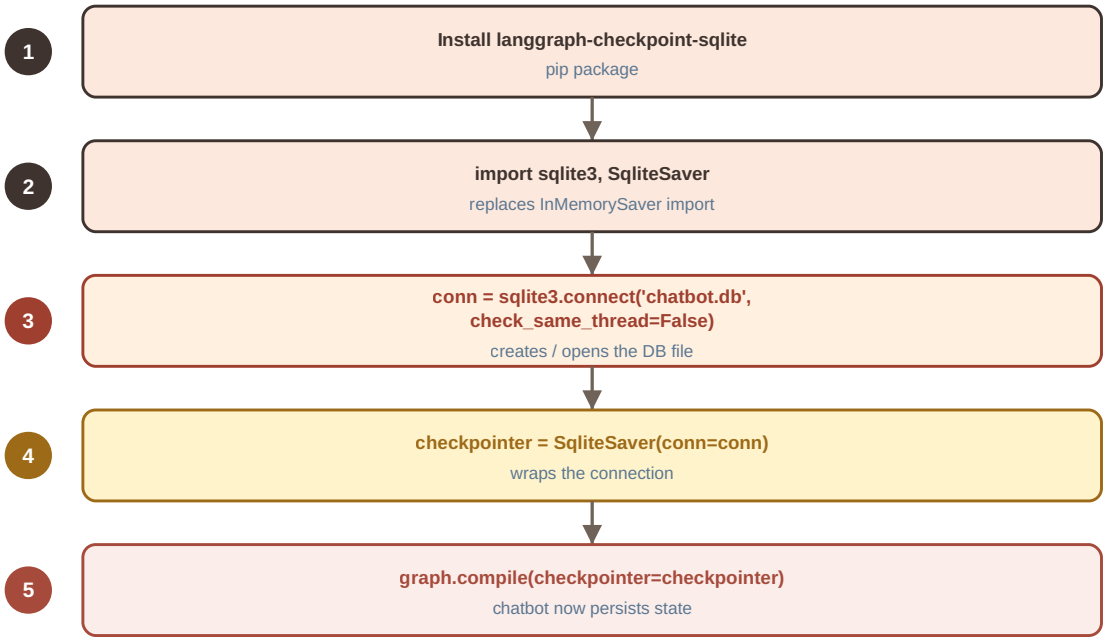

Figure 14.2 — Five steps to replace InMemorySaver with a SQLite-backed checkpointer.

1. INSTALL THE PACKAGE

```
# SqliteSaver is not bundled with core LangGraph —
# it is a separate community package (may be merged into core in a future version)
pip install langgraph-checkpoint-sqlite
```

2. SWAP THE IMPORTS

BEFORE — INMEMORYSAVER

```
from langgraph.checkpoint.memory import
InMemorySaver
```

AFTER — SQLITESAVER

```
from langgraph.checkpoint.sqlite import
SqliteSaver
import sqlite3
```

3 & 4. CREATE THE DATABASE CONNECTION, THEN THE CHECKPOINTER

```
# Creating Database
conn = sqlite3.connect(database = 'chatbot.db', check_same_thread = False)

# Checkpointer
checkpointer = SqliteSaver(conn = conn)
```

Two lines of code do all the work: `sqlite3.connect(...)` creates the `chatbot.db` file (in the project directory) if it does not already exist and returns a connection object; `SqliteSaver(conn=conn)` wraps that connection so LangGraph can read and write checkpoint state through it.

> **COMMON MISTAKE**
>
> Leaving `check_same_thread` at its default value of `True` causes a runtime error once the app uses multiple threads (which it does — each conversation gets its own `thread_id`). SQLite by default only allows a connection to be used by the exact thread that created it. Setting `check_same_thread=False` explicitly disables that restriction so the same database connection can be shared safely across the app's different conversation threads.

5. COMPILE THE GRAPH WITH THE NEW CHECKPOINTER

```
chatbot = graph.compile(checkpointer = checkpointer)
```

From this point on, the rest of the graph-building code (state definition, nodes, edges) is completely unchanged — only the checkpointer object passed into `compile()` is different.

> ↻ **Quick Revision — SQLite Setup**
>
> • Install with: pip install langgraph-checkpoint-sqlite.
>
> • Import SqliteSaver from langgraph.checkpoint.sqlite, plus Python's built-in sqlite3.
>
> • sqlite3.connect('chatbot.db', check_same_thread=False) creates/opens the DB file and returns a connection.
>
> • check_same_thread=False is required because the app uses multiple conversation threads sharing one connection.
>
> • SqliteSaver(conn=conn) wraps the connection; pass it to graph.compile(checkpointer=checkpointer).

4. Verifying Persistence Across Restarts

Example

To prove persistence actually works, a small manual test invokes the compiled chatbot directly (bypassing Streamlit), using a fixed `thread_id`:

```
CONFIG = {'configurable': {'thread_id': 'thread-1'}}

response = chatbot.invoke(
    {'messages': [HumanMessage(content = "Hi my name is Nitish")]},
    config = CONFIG
)

print(response)
```

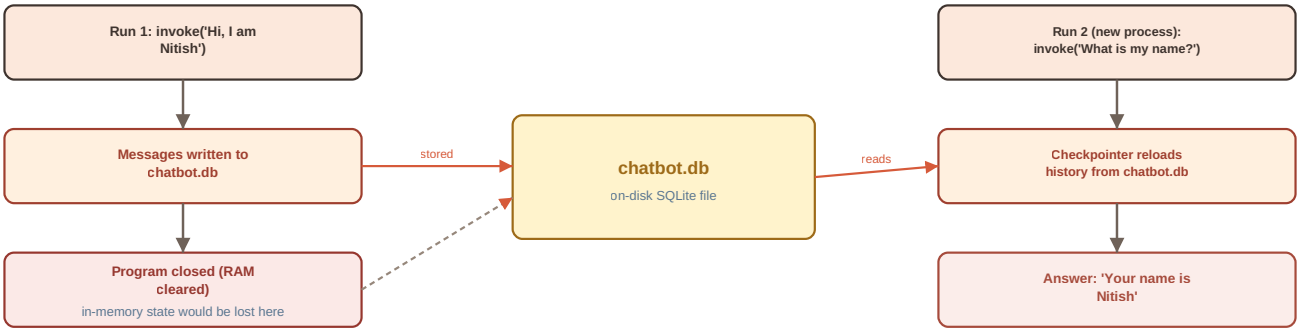

Figure 14.3 — Program state survives a full process restart because it is read back from chatbot.db.

Running this script the first time produces the reply and, importantly, creates a new `chatbot.db` file in the project folder. Running the script a *second time* (a brand-new process, with the same `thread-1`) and asking *"What is my name?"* returns **"Your name is Nitish"** — proving the checkpointer reloaded the earlier conversation from disk rather than from RAM.

> **REAL WORLD INSIGHT**
>
> With the old InMemorySaver, this same test would fail on the second run — the process's memory would be empty and the chatbot would have no idea who "Nitish" is. The only difference between the two behaviors is the checkpointer backend; nothing about the graph, nodes, or edges changed.

Thread Isolation

Switching to a different `thread_id` (e.g. `thread-2`) and sending *"Hi my name is Rahul"* starts a completely separate conversation inside the *same* database file. Asking *"What is my name?"* under `thread-1` still correctly answers **Nitish**, while the same question under `thread-2` answers **Rahul**.

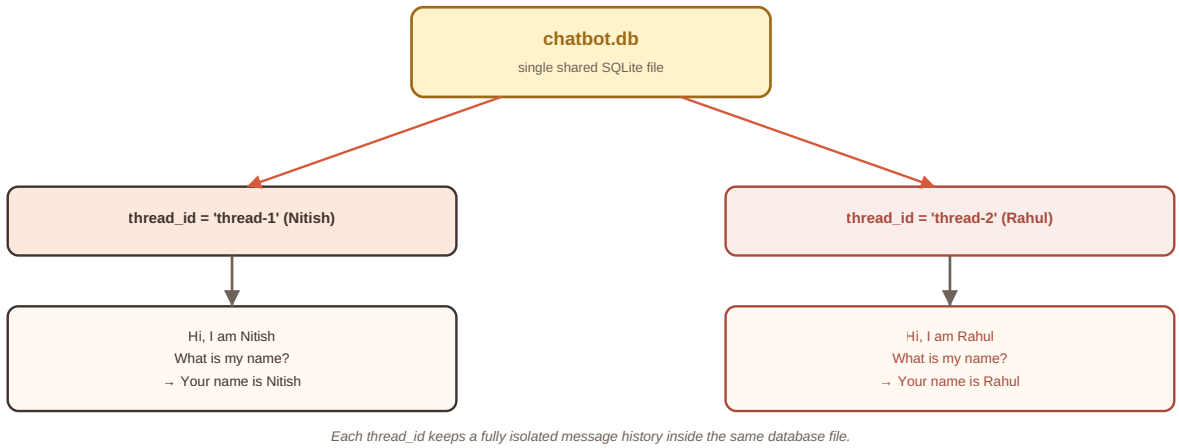

Figure 14.4 — Multiple threads keep fully isolated histories inside one shared SQLite file.

↻ Quick Revision — Persistence & Isolation

- A process can be closed and restarted, and chatbot.invoke() with the same thread_id still returns full history.
- Persistence is verified by asking a follow-up question ('What is my name?') in a fresh process run.
- Each thread_id keeps a fully isolated message history, even though all threads live in the same chatbot.db file.
- Switching thread_id switches which isolated conversation the chatbot is answering from.

5. Inspecting the Database & Understanding Checkpoint Counts

How It Works

Every single call to `chatbot.invoke()` or `chatbot.stream()` does not create just one checkpoint — it creates **three**: one at `START`, one after the graph's node executes, and one at `END`. This matches the checkpointing behavior covered earlier in the persistence-fundamentals video in this playlist.

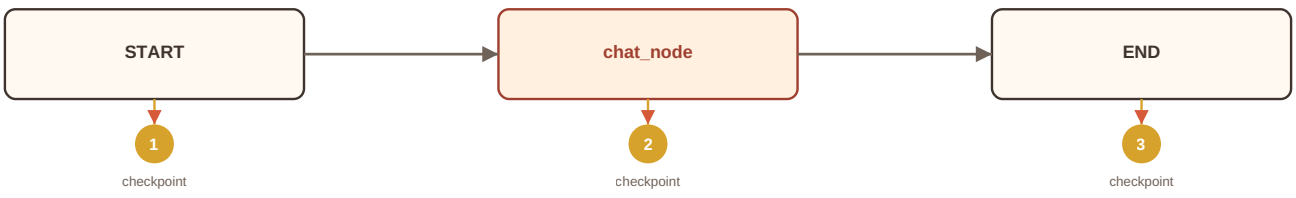

Figure 14.5 — A single invoke() call produces three checkpoints: START, chat_node, END.

REAL WORLD INSIGHT

This is why inspecting `chatbot.db` after a handful of turns shows a much larger checkpoint count than the number of messages sent. In the lecture's demo, 6 user turns across 2 threads produced 18 total checkpoints (9 per thread) — 3 checkpoints per invocation, exactly as expected.

Viewing the Database Visually

A raw `.db` file is not human-readable on its own. The recommended way to inspect it during development is a VS Code extension such as **SQLite Viewer** (publisher: Florian Klampfer). Once installed, clicking the `chatbot.db` file opens a browsable table view directly inside the editor, showing every checkpoint row, its associated `thread_id`, and the serialized state.

> **BEST PRACTICE**
>
> Install a SQLite viewer extension early when working with a file-based checkpointer. Being able to visually confirm how many checkpoints exist per thread builds confidence that persistence is actually working, and makes it far easier to debug unexpected message counts.

↻ Quick Revision — Checkpoint Counts

- Every invoke()/stream() call creates 3 checkpoints: START, node execution, END.
- Total checkpoints in the DB = 3 x number of invocations, split per thread_id.
- A raw .db file is not readable in plain text; use a viewer (e.g. VS Code's 'SQLite Viewer' extension).
- Viewing the DB confirms both the checkpoint count and that different threads store separate message sets.

6. Retrieving All Threads (retrieve_all_threads)

Why It Exists

The frontend needs to show a sidebar list of past conversations the moment the app loads — not just the threads created during the current session. That means the backend needs a function that asks the database directly: *"which unique thread_ids already exist?"*

How It Works

The checkpointer object exposes a `.list(config)` method. Passing `None` as the config returns a generator over *every* checkpoint currently stored (not filtered to one thread). Each checkpoint tuple carries a `.config` dictionary containing `configurable.thread_id`.

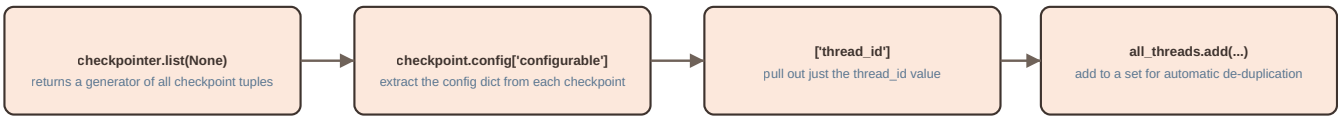

Figure 14.6 — Extracting unique thread_ids from checkpointer.list(None).

```python
# Extracting all the threads from the database

def retrieve_all_threads():
    all_threads = set()

    for checkpoint in checkpointer.list(None):
        all_threads.add(checkpoint.config['configurable']['thread_id'])

    return list(all_threads)
```

Because `all_threads` is a **set**, duplicate `thread_id` values (recall: 3 checkpoints per thread per invocation) automatically collapse into a single unique entry per thread. The function returns a plain `list` of unique thread IDs, ready to seed the frontend's session state.

> **INTERVIEW TIP**
>
> If asked why a `set` is used instead of a `list` while collecting thread_ids, explain that a set enforces uniqueness automatically and in O(1) average time per insertion, avoiding the need for manual `if x not in list` checks while iterating over every checkpoint row.

> ↻ **Quick Revision — retrieve_all_threads()**
>
> • checkpointer.list(None) returns a generator of every checkpoint in the database.
>
> • Each checkpoint's config['configurable']['thread_id'] identifies which thread it belongs to.
>
> • Adding thread_ids to a Python set automatically de-duplicates repeated values.
>
> • retrieve_all_threads() returns list(all_threads): the unique thread IDs currently stored in chatbot.db.

7. Wiring the Frontend to Load Existing Threads

Problem It Solves

In the Chapter 13 frontend, `chat_threads` in session state was always initialized as an **empty list** — which made sense when there was no permanent storage: every fresh load genuinely had zero prior threads. Now that persistence exists, initializing to an empty list would discard the very history the chapter just worked to preserve.

How It Works

The fix touches exactly one place in the Streamlit session-setup block: instead of initializing `chat_threads` to `[]`, it is initialized by calling `retrieve_all_threads()` from the new database-backed backend module.

BEFORE — CHAPTER 13 (EMPTY LIST)

```
from langgraph_backend import chatbot

if 'chat_threads' not in st.session_state:
    st.session_state['chat_threads'] = []
```

AFTER — CHAPTER 14 (LOADED FROM DB)

```
from langgraph_database_backend import chatbot,
retrieve_all_threads

if 'chat_threads' not in st.session_state:
    st.session_state['chat_threads'] =
retrieve_all_threads()
```

Everything downstream — the sidebar loop that renders a button per thread, `add_thread()`, `reset_chat()`, and `load_conversation()` — is unchanged from Chapter 13. Because the sidebar always iterates over whatever is in `st.session_state['chat_threads']`, populating that list correctly at startup is the only change required for old threads to reappear automatically.

IMPORTANT

Persistence is a *backend* concern (which checkpointer is compiled into the graph) combined with a *one-line frontend bootstrap fix* (how `chat_threads` is initialized). The frontend's message-rendering, streaming, and thread-switching logic never needed to change — it was already written generically enough to work with any list of thread IDs, whether freshly created or reloaded from disk.

↻ Quick Revision — Frontend Wiring

- Only one line changes in the frontend: chat_threads is initialized from retrieve_all_threads() instead of [].
- The import source also changes to the new database-backed backend module.
- Sidebar rendering, add_thread(), reset_chat(), and load_conversation() are unchanged from Chapter 13.
- Result: on every fresh app load, all previously stored threads (and their full history) reappear automatically.

K. Key Takeaways

- **InMemorySaver is RAM-only** — all conversation history is lost when the app closes or the page reloads, since nothing is ever written to disk.
- **LangGraph offers three checkpointer backends**: InMemorySaver (RAM), SqliteSaver (file-based SQLite, used in this chapter for prototyping), and PostgresSaver (production-grade database).
- **SqliteSaver setup** requires the external package langgraph-checkpoint-sqlite, a sqlite3 connection created with check_same_thread=False, and passing that connection into SqliteSaver(conn=conn).
- **check_same_thread=False** is required because SQLite by default restricts a connection to the thread that created it, and this app deliberately uses multiple conversation threads.
- **Every invoke()/stream() call produces 3 checkpoints** — at START, after node execution, and at END — which explains why checkpoint counts look larger than message counts.

- **Thread isolation is preserved**: multiple thread_ids share a single database file but never mix message histories.
- **retrieve_all_threads()** uses checkpointer.list(None) plus a Python set to extract the unique thread_ids currently stored in the database.
- **The only frontend change needed** is initializing chat_threads from retrieve_all_threads() instead of an empty list — every other UI function from Chapter 13 works unmodified.

R. Revision Sheet

InMemorySaver

RAM-based checkpointer; loses all thread state when the process ends. Suitable only for short-lived testing.

SqliteSaver

File-based checkpointer backed by a local SQLite database (e.g. chatbot.db). Persists across restarts; ideal for prototyping/learning.

PostgresSaver

Production-grade checkpointer backed by a full PostgreSQL database, used for real deployed applications.

check_same_thread=False

SQLite connection flag that disables SQLite's default single-thread restriction, required whenever a connection is shared across multiple conversation threads.

Checkpoint Triplet

Each invoke()/stream() call writes 3 checkpoints: START, node execution, END — explaining higher-than-expected checkpoint counts.

retrieve_all_threads()

Backend helper that calls checkpointer.list(None), extracts thread_id from each checkpoint's config, and returns the de-duplicated (set-based) list of thread IDs.

Thread Isolation

Multiple thread_ids can share one SQLite file while keeping their message histories completely separate and independently retrievable.

Frontend Bootstrap Fix

chat_threads is initialized from retrieve_all_threads() instead of an empty list, so previously stored conversations reappear automatically on every app load.

I. Interview Questions

Q1. Why does InMemorySaver fail to preserve chat history across application restarts?

Q2. What are the three checkpointer backends supported by LangGraph, and when would you choose each one?

Q3. What is the purpose of the check_same_thread=False parameter when creating a SQLite connection for a checkpointer, and what error does omitting it cause?

Q4. Walk through the exact two lines of code required to create a SqliteSaver-backed checkpointer from a raw SQLite connection.

Q5. Why does a single chatbot.invoke() call typically create three checkpoints instead of one?

Q6. How does retrieve_all_threads() guarantee that returned thread_ids are unique, and why is that data structure chosen?

Q7. Explain how two different thread_ids can share the same underlying SQLite database file without their conversation histories mixing.

Q8. What is the single change required in the Streamlit frontend to make previously stored conversations appear in the sidebar on a fresh app load?

Q9. Why is SqliteSaver considered appropriate for prototyping but not typically recommended for production systems?

Q10. How would you practically verify, without reading application logs, that a chatbot's persistence layer is actually writing to disk?

F. Further Reading

- LangGraph Persistence documentation — official concepts guide covering checkpointers, threads, and state.
- langgraph-checkpoint-sqlite package documentation (PyPI) — installation and SqliteSaver API reference.
- langgraph-checkpoint-postgres package documentation — for production-grade persistence with PostgreSQL.
- Python sqlite3 standard library documentation — connection objects, threading behavior, and check_same_thread.
- SQLite Viewer extension (VS Code Marketplace, publisher: Florian Klampfer) — for visually inspecting .db files during development.